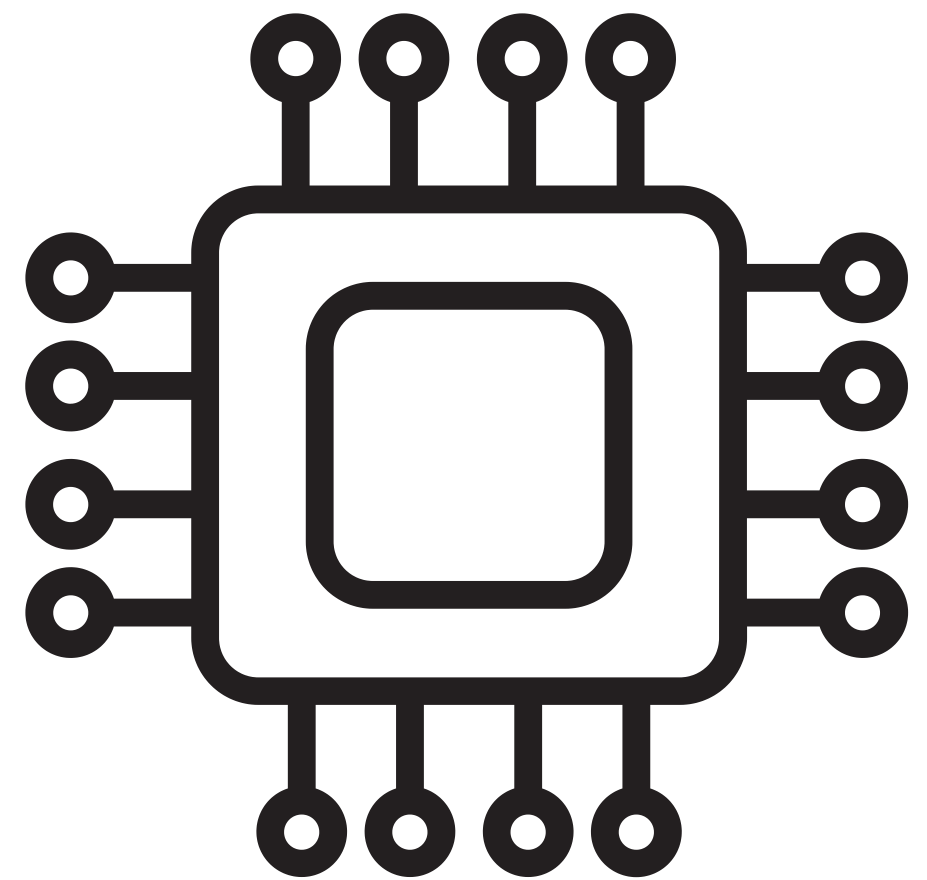


ZOOM Digital - Etapa 3: Desenvolvimento da Aplicação em C + Interação com Mouse + Coprocessador



TEC499 - SISTEMAS DIGITAIS

DISCENTES: ÍTALLO GUIMARÃES, KEVIN BORGES E LUCAS SILVA



Desenvolvimento no projeto – Etapa 3

Desenvolver a aplicação principal em C para interação com o usuário;

Integrar a aplicação com a API em Assembly para comunicação HPS ↔ FPGA;

Processar a janela selecionada via coordenadas do mouse aplicando algoritmos de zoom-in e zoom-out via teclado;

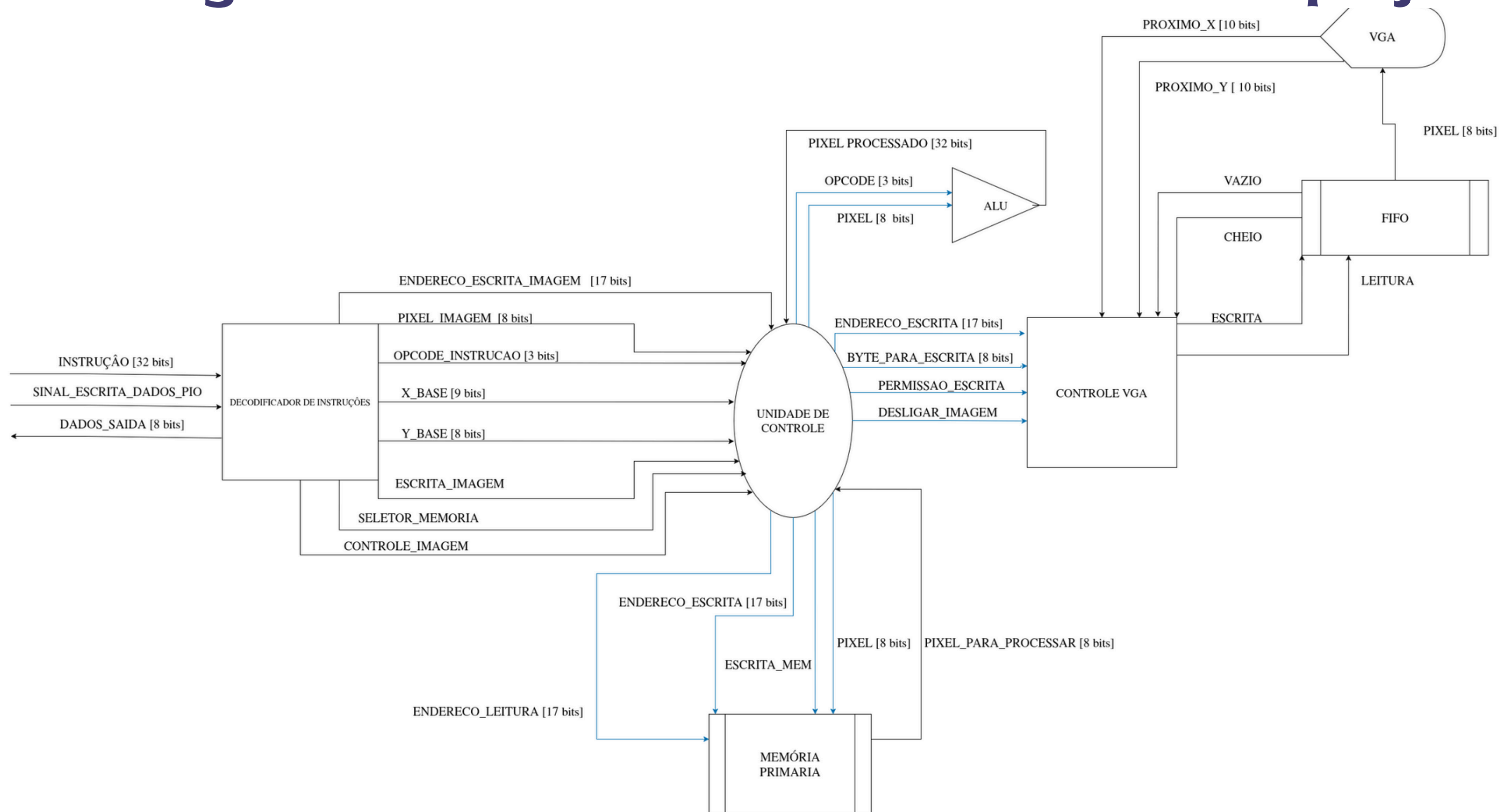
Garantir a exibição contínua da imagem processada via VGA.

Sumário de Apresentação

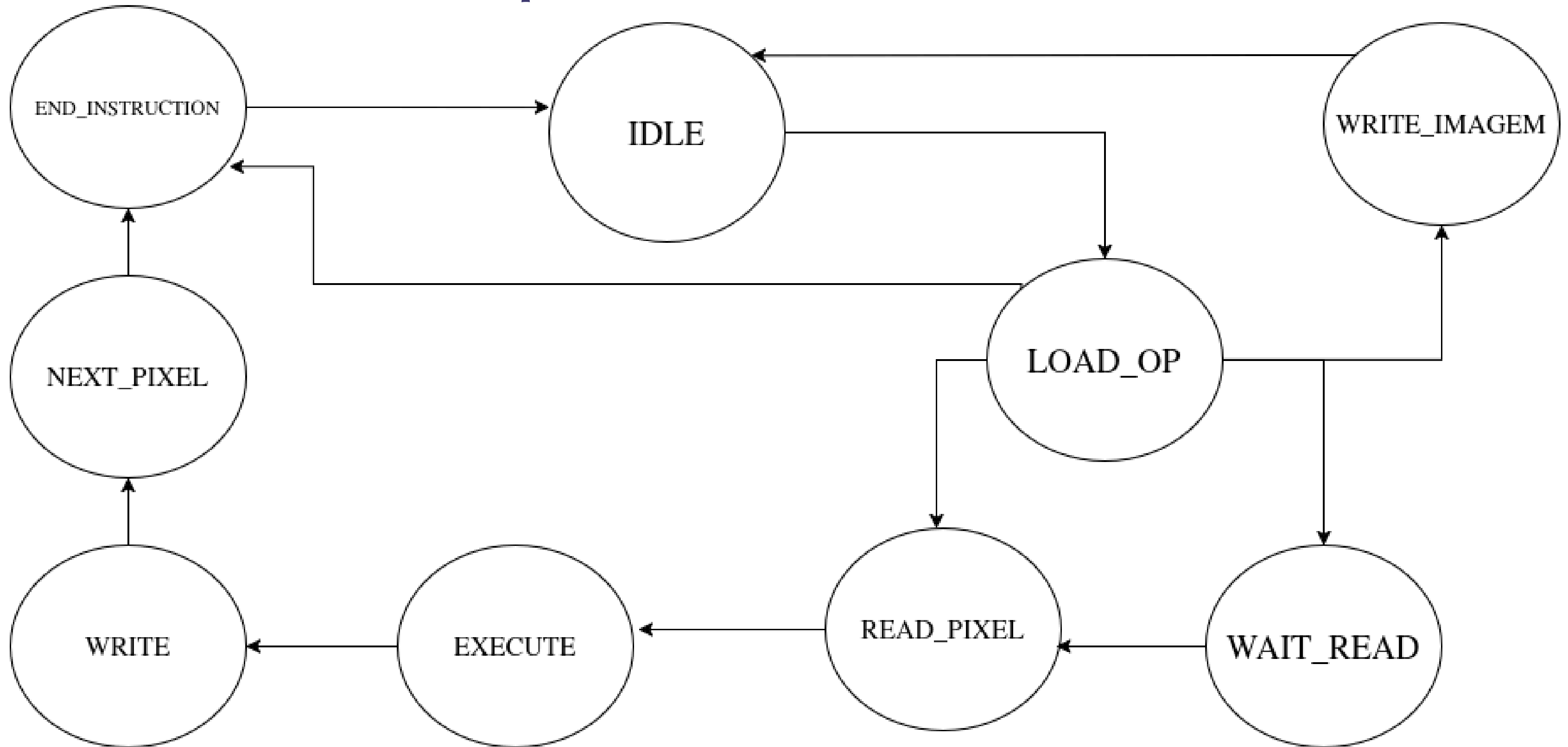
- Diagrama de conexão entre os módulos;
- Máquina de Estados Geral;
- ISA's;
- Funções principais da API;
- Seleção da Janela com o Mouse;
- Arquitetura Geral do Sistema;
- Estrutura de Arquivos;
- Roteiro de testes.



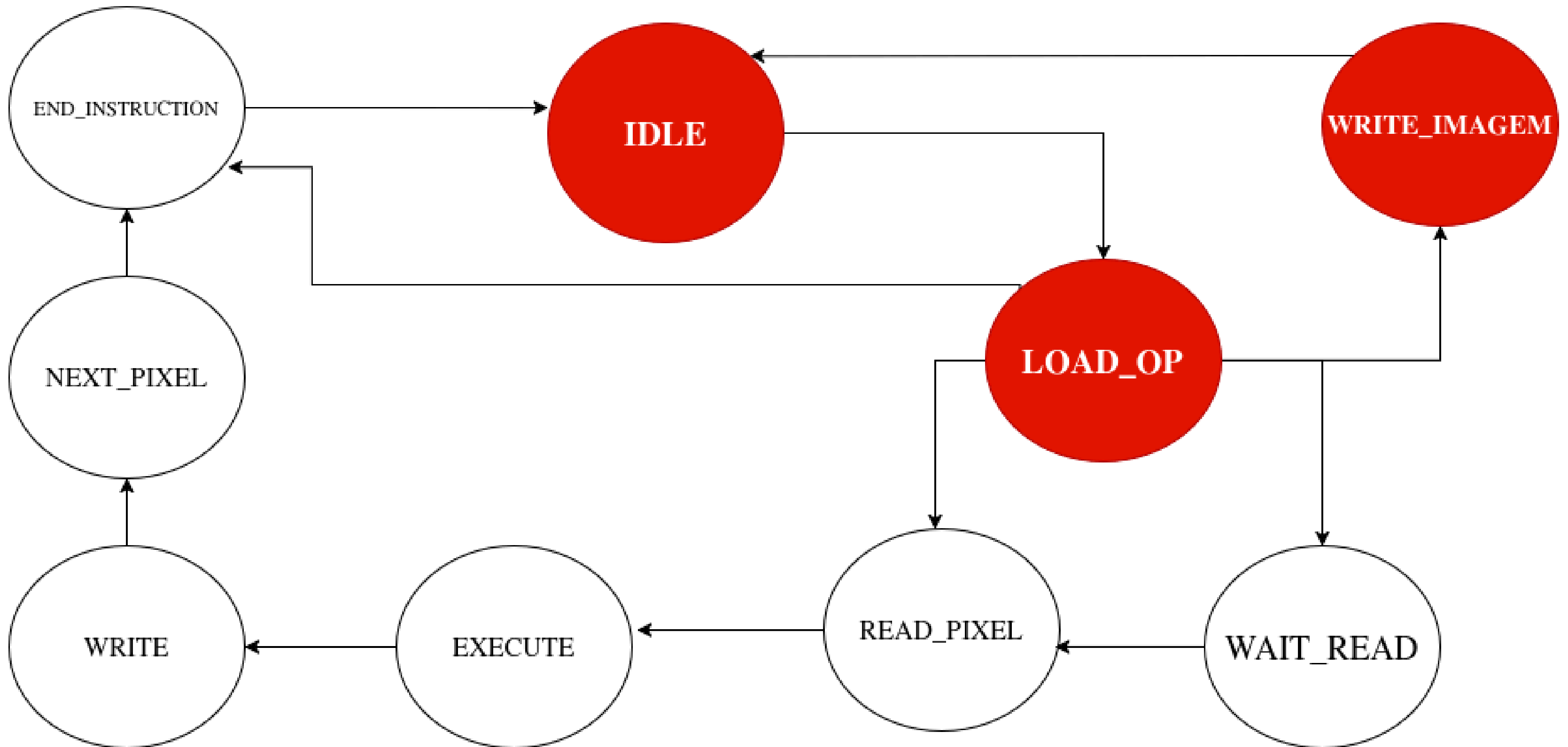
Diagrama de conexão entre os módulos do projeto



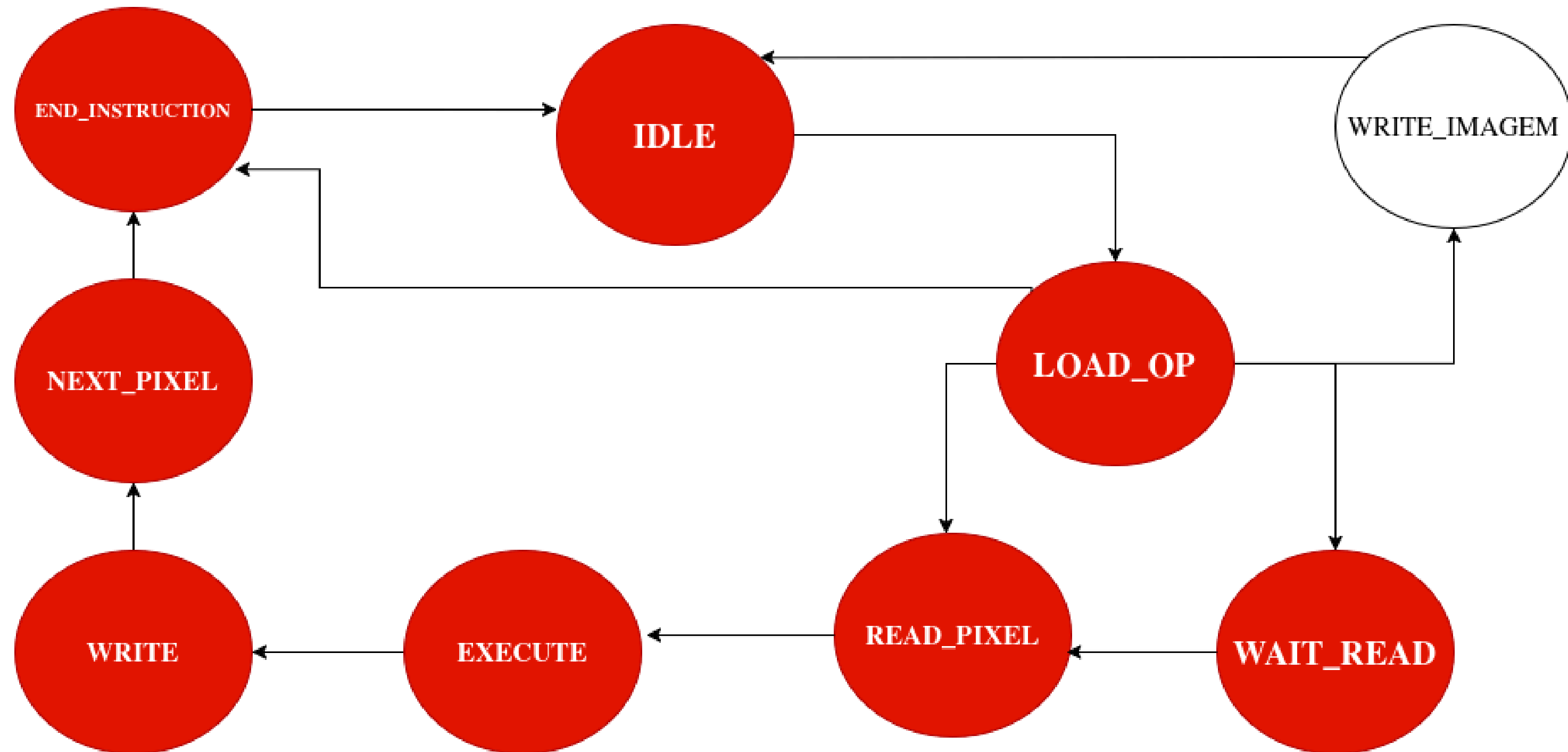
Máquina de Estados Geral



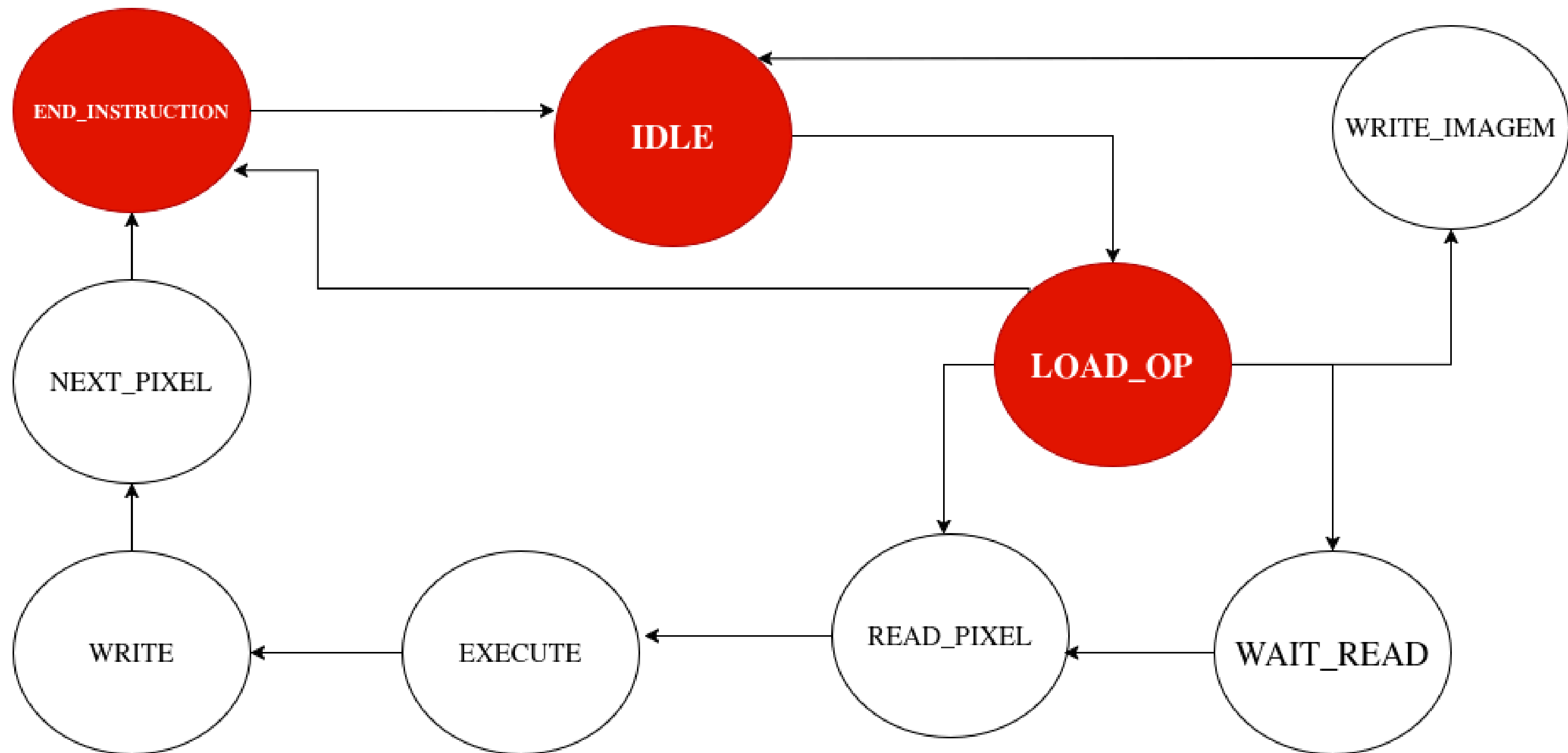
Máquina de Estados - Enviar pixel



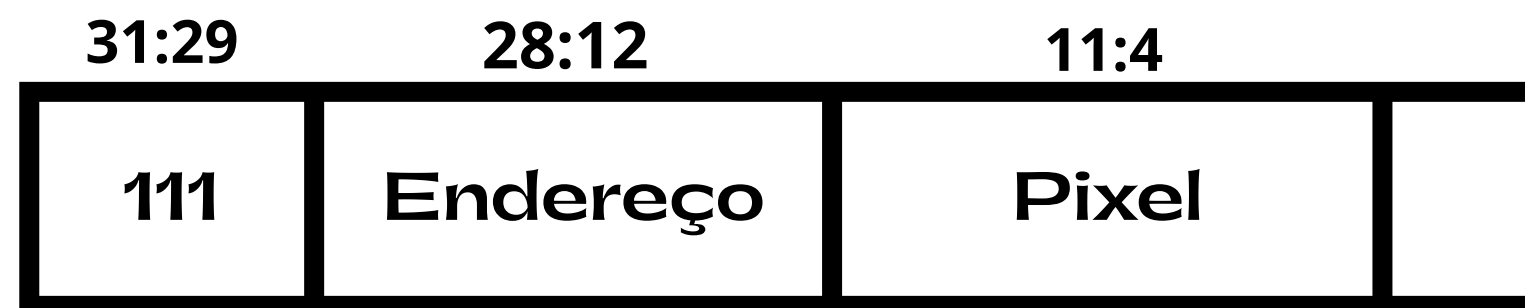
Máquina de Estados - Carregar Pixel



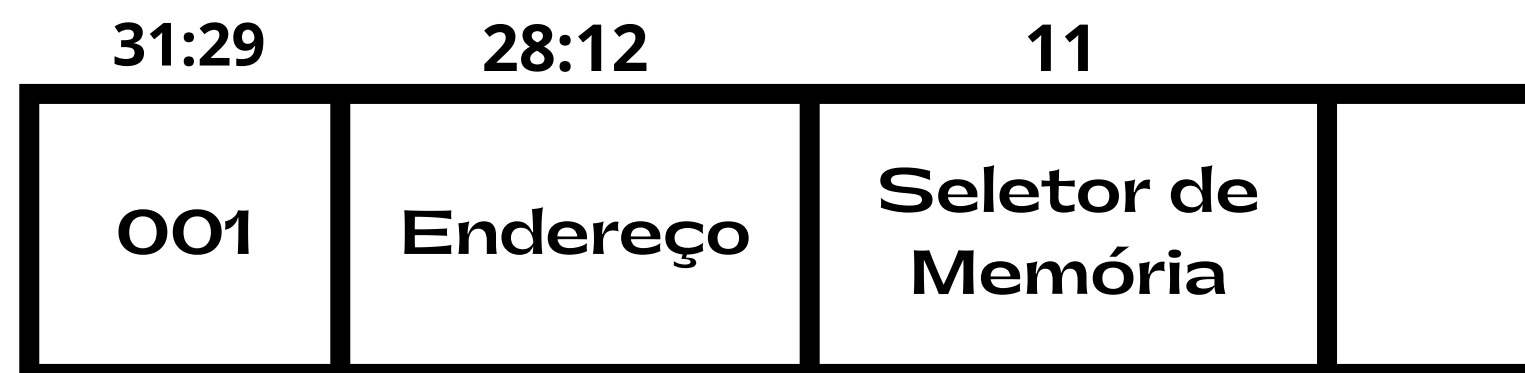
Máquina de Estados - Controle imagem



ISA: Enviar Pixel

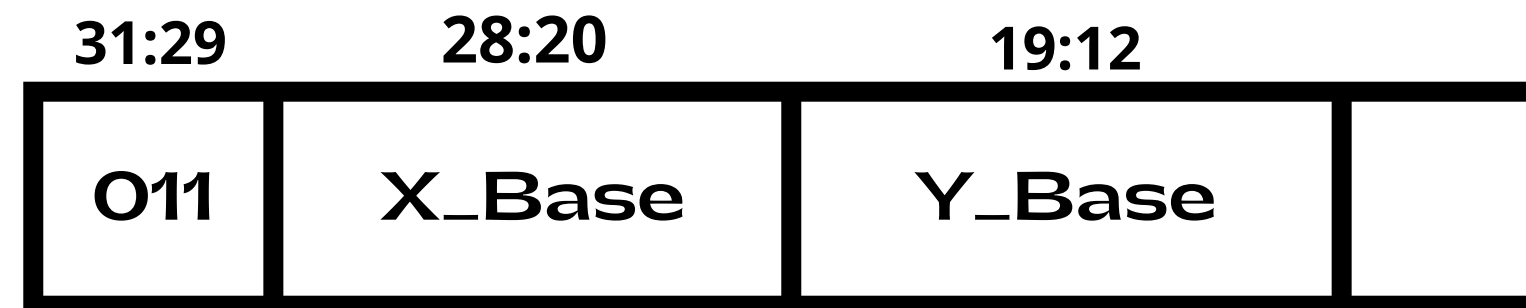


ISA: Carregar Pixel

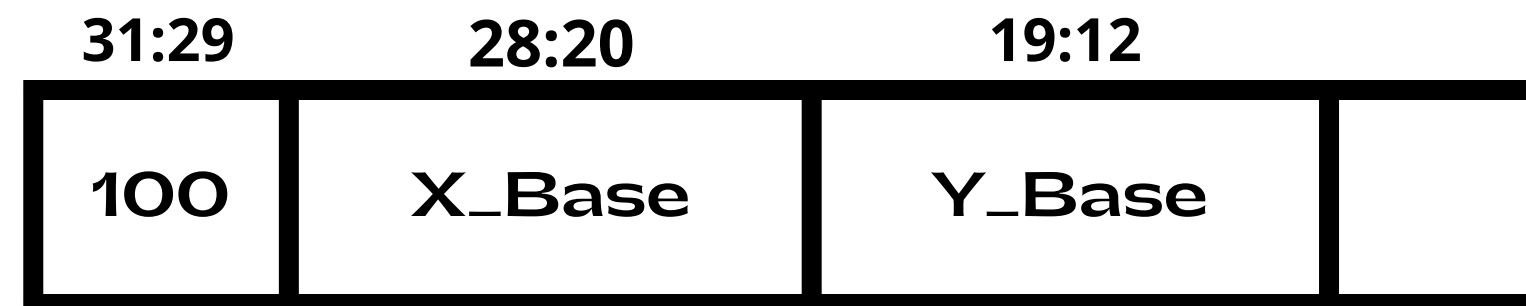


ISA: Algoritmos de Zoom-In

Vizinho Mais Próximo:



Replicação de Pixel:



ISA: Controle de Imagem

31:29

0

010		Desativar Imagem
-----	--	------------------

Funções Principais da API				
<u>Opcode (bin)</u>	<u>Nome</u>	<u>Função</u>	<u>Parâmetro</u>	<u>Retorno</u>
111	ENVIAR_IMAGEM_FPGA	Copiar todos os pixels da imagem PGM para a memória de vídeo da FPGA (76800 pixels).	char *imagem (array de pixels)	void
111	ENVIAR_PIXEL	Enviar um pixel individual (utilizado internamente por ENVIAR_IMAGEM_FPGA).	int pixel, int endereco	void
1	CARREGAR_PIXEL	Ler um pixel da memória da FPGA (RAM da imagem).	int endereco, int memoria_exibicao	int (valor do pixel lido)
10	CONTROLE_IMAGEM	Alternar imagem ativa (exibir imagem original ou imagem processada).	int desligar (0 ou 1)	void
11	VIZINHO_MAIS_PROXIMO	Zoom In usando interpolação Nearest Neighbor.	int x, int y (coordenadas da janela)	void
100	REPLICACAO_PIXEL	Zoom In por replicação direta de pixels.	int x, int y	void
101	DECIMACAO	Zoom Out por amostragem / decimação.	void	void
110	MEDIA_DE_BLOCOS	Zoom Out calculando a média de blocos 2×2.	void	void

Seleção da Janela com o Mouse

Como funciona:

- Leitura do mouse pelo dispositivo `/dev/input/eventX`.
- Tratamento de eventos (`EV_REL`, `BTN_RIGHT`).
- Usuário marca:
 - Primeiro canto da janela
 - Segundo canto (diagonal oposta)

O que o sistema faz:

- Mostra coordenadas em tempo real.
- Valida limites da janela.
- Armazena $(x1, y1)$ e $(x2, y2)$.

Algoritmo de Zoom na Janela

Passos Detalhados do Algoritmo:

1. Seleção da janela

O usuário escolhe dois pontos com o mouse:

- $(x1, y1)$
- $(x2, y2)$

A partir deles calcula-se:

- $W = |x2 - x1|$ (largura da janela)
- $H = |y2 - y1|$ (altura da janela)

2. Carregamento da Imagem Original

O sistema lê a imagem da memória 0 (memória de trabalho da FPGA).

Esta é a imagem original, sem modificações.

Algoritmo de Zoom na Janela

3. Criação do “Buffer da Janela”

Um novo buffer é criado em software, representando toda a imagem, porém:

- Somente a região selecionada é copiada da imagem original.
- Todo o resto é preenchido com preto (0x00).

Resultado:

Um “recorte” da janela isolado em um fundo preto.

Este buffer é então enviado para a FPGA usando ENVIAR_PIXEL.

Algoritmo de Zoom na Janela

4. Aplicação do Zoom (Zoom In / Zoom Out)

Com o buffer em memória, o software envia o opcode do zoom escolhido:

- Replicação de Pixel
- Vizinho Mais Próximo (Nearest Neighbor)
- Média de Blocos
- Decimação

Ao receber o comando:

- A FPGA processa apenas os pixels não-pretos (a região da janela).
- O resultado é escrito na memória 1 (memória de resultado).

5. Leitura da Imagem Processada

Após o processamento, o software:

- Lê a subimagem ampliada da memória 1,
- Traz esta área de volta para o software via `CARREGAR_PIXEL()`.

Este passo extrai somente a parte ampliada.

Algoritmo de Zoom na Janela

6. Combinação (Overlay) na Imagem Original

A imagem original é usada como base.

Por cima dela:

- A área ampliada da janela é desenhada exatamente na posição original.
- O restante da imagem permanece igual.

Resultado visual:

A área selecionada se comporta como uma lupa digital.

7. Envio da Imagem Final ao VGA

A imagem combinada final é então:

- Escrita novamente no buffer principal
- Enviada ao VGA usando ENVIAR_PIXEL

Isto atualiza a tela imediatamente.

Arquitetura Geral do Sistema

Funções principais do teclado

- **Setas (↑ ↓ ← →)**
 - Navegam pelos menus
 - Movem a janela no modo interativo de zoom
 - Atualizam a posição da janela e reaplicam o zoom automaticamente
- **Tecla “+” (mais)**
 - Aplica Zoom In
 - Se houver janela ativa: calcula, envia, processa e renderiza o zoom na região
 - Se não houver janela: aplica zoom global
 - No modo interativo: aumenta velocidade de movimentação da janela
- **Tecla “-” (menos)**
 - Aplica Zoom Out
 - Reverte o Zoom In anterior (janela ou global)
 - No modo interativo: reduz velocidade de movimentação da janela
- **Tecla “q”**
 - Sai do modo interativo da janela
 - Retorna ao menu principal

Arquitetura Geral do Sistema

Componentes envolvidos:

- Aplicação em C (HPS).
- Driver/Biblioteca em Assembly (biblioteca.s).
- Coprocessador FPGA (operações de zoom).
- Interface com VGA.

Fluxo:

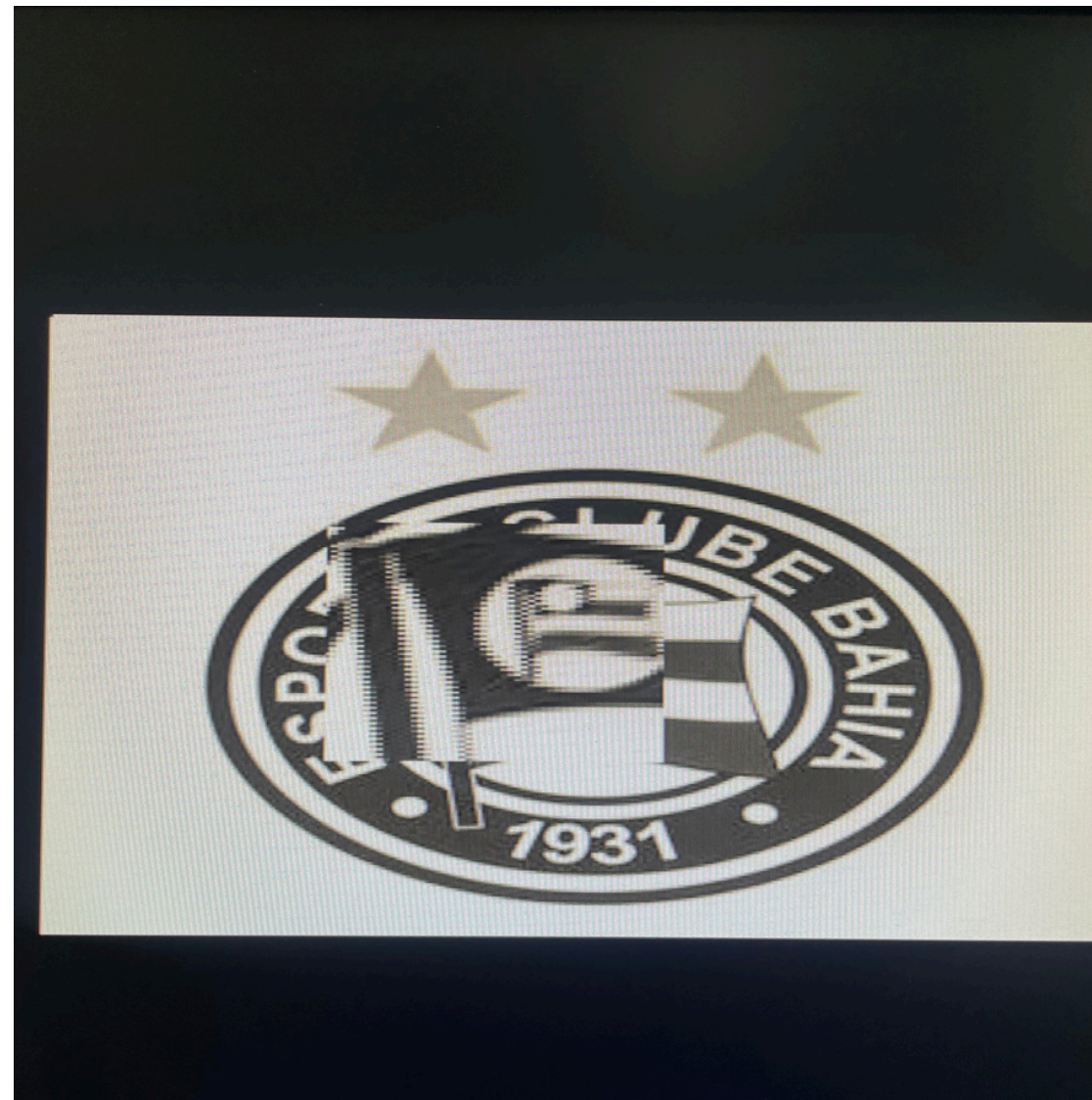
Usuário → Menu C → Driver Assembly → Coprocessador FPGA → Imagem no VGA

Estrutura de Arquivos da Fase 3

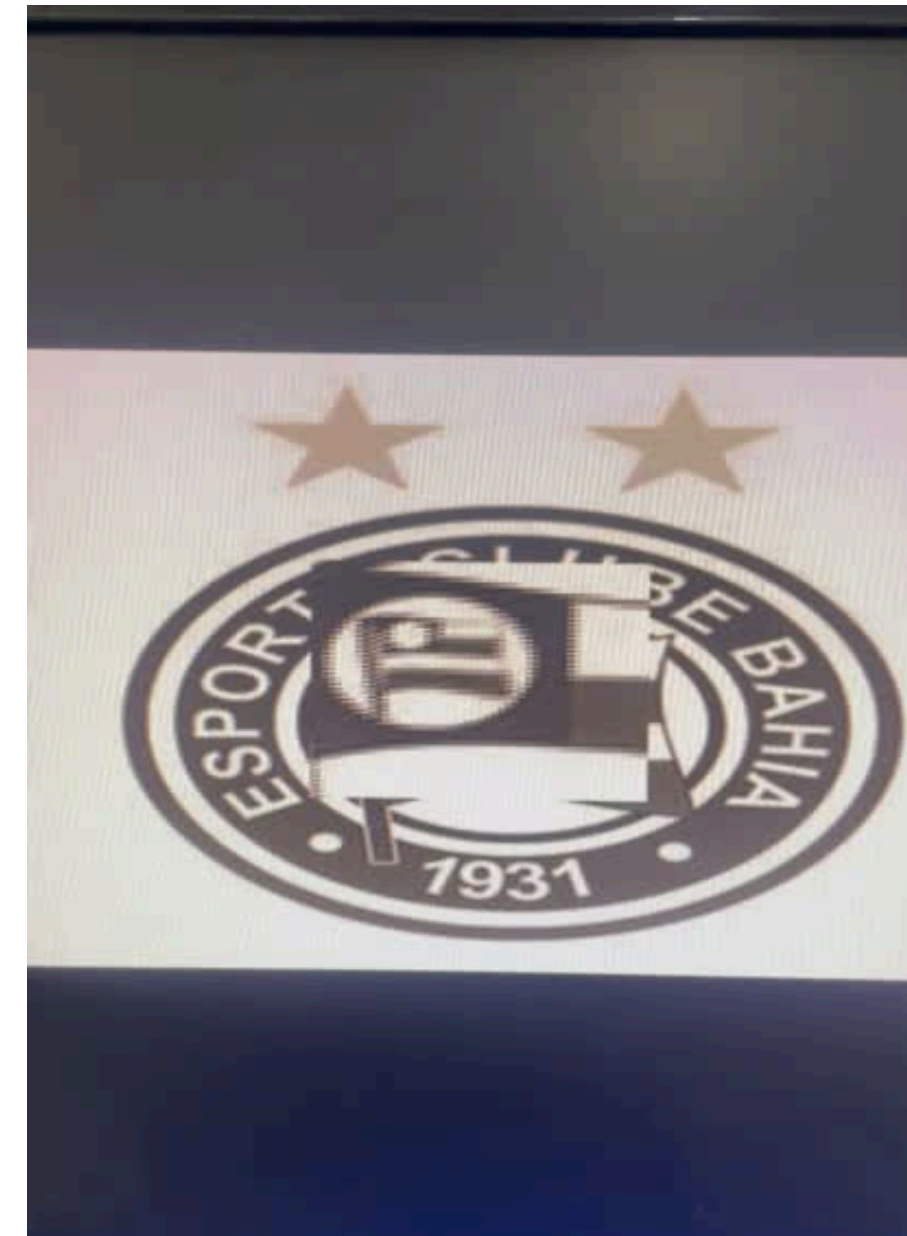
- **api/main.c** → Aplicação principal em C.
- **api/api.h** → Header com protótipos.
- **api/biblioteca.s** → Driver e rotinas de controle.
- **api/makefile** → Compilação e execução.
- **api/*.pgm** → Imagens em escala de cinza.

Roteiro de Testes

Janela em destaque

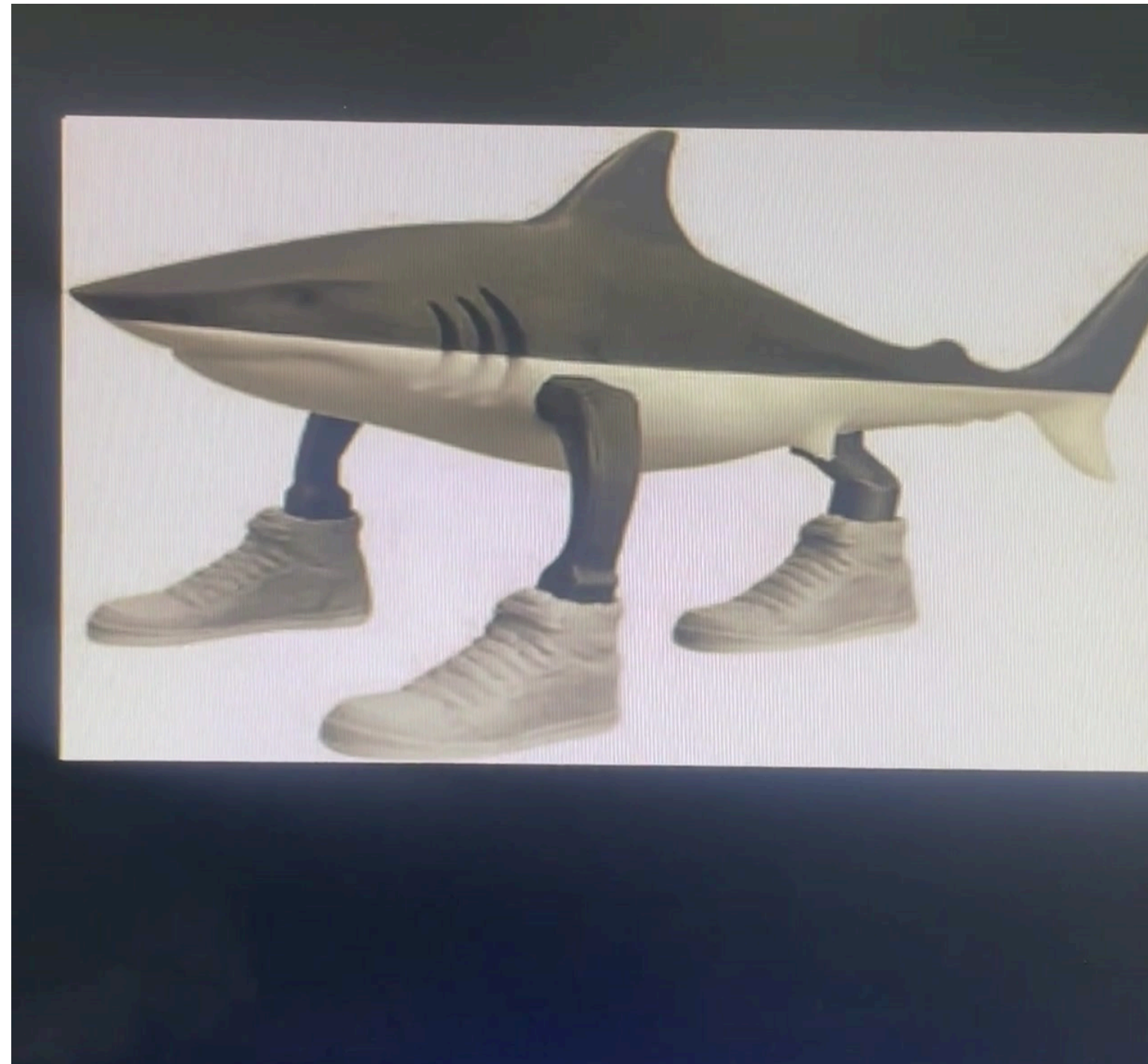


Movimentação da janela na tela



Roteiro de Testes

Demonstração de uso do menu



Conclusão

A Etapa 3 refinou a integração software + hardware de modo que agora todo e qualquer usuário consegue utilizar o projeto de forma intuitiva;

HPS controla completamente o coprocessador;

Os requisitos propostos foram atendidos

Referências:

PATTERSON, David A.; HENNESSY, John L.
Computer organization and design ARM edition: the
hardware software interface. Morgan kaufmann, 2016.

Manual da placa disponível em:
<https://drive.google.com/file/d/1dBaSfXi4GcrSZ0JlzRh5iixaWmqOgo2j/view>